

To complete **Stage 0: create SQLite database**, you need to implement persistent database storage using SQLite instead of the in-memory array or list.

Since your project includes both a JavaScript (Node.js) and a Python (FastAPI) lane, complete implementation instructions for both options are provided below. **Choose the code block that matches your project lane.**

● Node.js Lane (Express + better-sqlite3)

Update your `server.js` file to initialize the database connection, automatically generate the `tasks` table, and seed it only if it is empty.

javascript

```
const express = require('express');
const Database = require('better-sqlite3');
const app = express();
const PORT = 3000;

app.use(express.json());

// Open (and automatically create) the tasks.db SQLite file
const db = new Database('tasks.db');

// Create the tasks table if it does not exist
db.prepare(`
  CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    done INTEGER DEFAULT 0
  )
`).run();

// Check if the table is empty before seeding example data
const rowCount = db.prepare('SELECT COUNT(*) as count FROM tasks').get();

if (rowCount.count === 0) {
  const insertTask = db.prepare('INSERT INTO tasks (title, done) VALUES (?, ?)');

  // Seed the 3 default tasks
  insertTask.run("Buy groceries", 0);
```

```
insertTask.run("Clean the room", 1);
insertTask.run("Study API design", 0);

console.log("Database seeded successfully with initial tasks.");
}

// --- Endpoints ---

app.get('/', (req, res) => {
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks", "/stats"] });
});

app.get('/health', (req, res) => {
  res.json({ status: "ok" });
});

// GET /tasks: Updated to fetch records straight from the SQLite database
app.get('/tasks', (req, res) => {
  const { done, search } = req.query;
  let query = 'SELECT * FROM tasks WHERE 1=1';
  const params = [];

  if (done !== undefined) {
    query += ' AND done = ?';
    params.push(done === 'true' ? 1 : 0);
  }

  if (search !== undefined && search.trim() !== "") {
    query += ' AND LOWER(title) LIKE ?';
    params.push(`%${search.toLowerCase()}%`);
  }

  const filteredTasks = db.prepare(query).all(...params);

  // Convert boolean database representations (0/1) back into actual booleans for JSON out
  const output = filteredTasks.map(t => ({ ...t, done: Boolean(t.done) }));
  res.json(output);
});

// Keep your remaining standard CRUD endpoints matching this structure...

app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```

Use code with caution.

● Python Lane (FastAPI + sqlite3 standard library)

Update your `main.py` file using Python's built-in `sqlite3` driver. This implementation initializes the database file, creates the structural constraints, and cleanly checks if seeding is necessary.

python

```
import sqlite3
from fastapi import FastAPI, HTTPException, status, Response, Query
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

DATABASE_FILE = "tasks.db"

def init_db():
    # Establishes connection and creates the file if missing
    conn = sqlite3.connect(DATABASE_FILE)
    cursor = conn.cursor()

    # Create target schema
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS tasks (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT NOT NULL,
            done INTEGER DEFAULT 0
        )
    """)
    conn.commit()

    # Check row counts first to prevent exponential multiplication loops
    cursor.execute("SELECT COUNT(*) FROM tasks")
    count = cursor.fetchone()[0]

    if count == 0:
        initial_tasks = [
            ("Buy groceries", 0),
```

```

        ("Clean the room", 1),
        ("Study API design", 0)
    ]
    cursor.executemany("INSERT INTO tasks (title, done) VALUES (?, ?)", initial_tasks)
    conn.commit()
    print("Database initialized and safely seeded.")

conn.close()

# Run the initialization procedure immediately on start
init_db()

# Custom helper function to handle connection pooling
def get_db_connection():
    conn = sqlite3.connect(DATABASE_FILE)
    conn.row_factory = sqlite3.Row # Enables access via dictionary keys
    return conn

# --- Endpoints ---

@app.get("/")
def read_root():
    return {"name": "Task API", "version": "1.0", "endpoints": ["/tasks", "/stats"]}

@app.get("/health")
def read_health():
    return {"status": "ok"}

@app.get("/tasks")
def read_tasks(done: Optional[bool] = Query(None), search: Optional[str] = Query(None)):
    conn = get_db_connection()
    cursor = conn.cursor()

    query = "SELECT * FROM tasks WHERE 1=1"
    params = []

    if done is not None:
        query += " AND done = ?"
        params.append(1 if done else 0)

    if search is not None and search.strip():
        query += " AND LOWER(title) LIKE ?"
        params.append(f"%{search.lower()}%")

```

```
cursor.execute(query, params)
rows = cursor.fetchall()
conn.close()

# Convert Row models to dictionary outputs with proper booleans
return [{"id": r["id"], "title": r["title"], "done": bool(r["done"])} for r in rows]
```

Use code with caution.

🎯 Checkpoint Verification Instructions

Follow these validation steps to confirm your database functions correctly:

1. **Boot and Shutdown Sequence:** Start your backend application server. Close it completely, restart it a second time, close it again, and start it a third time.
2. **Read Verification:** Open your terminal and issue a request to inspect the seed database array:

bash

```
curl -i http://localhost:3000/tasks
# (Or port 8000 depending on your runtime choice)
```

Use code with caution.

3. **Audit Check:** Verify that **exactly three tasks** return in the terminal array response. If you see six or nine items, your row-counting fallback logic failed to intercept duplicate database insertions.

💾 Git Version Control Commit

Once verification passes, run these commands sequentially in your terminal to save your progress:

bash

```
git add tasks.db server.js main.py # Stage your database file and updated code files
git commit -m "Stage 0: create SQLite database"
git push
```